

CY Land

Actor Framework - Projet Spring Boot

Thomas RYKACZEWSKI

Besma SAIDI

Paul PITIOT

Inès RIBAR

Myriam SAADI

Equipe 6

ING2 GIA1

CY Tech – Module JEE

Enseignante référente : Mme ZAOUCHE Djaouida

Année universitaire 2025–2026



Sommaire

- 1 Introduction
- 2 État de l'art
- 3 Présentation du Spring Actor Framework (SAF)
- 4 Architecture de l'application CY Land
- 5 Implémentation technique
- 6 Résultats et tests
- 7 Conclusion
- 8 Références et bibliographie



Introduction

- L'objectif du projet est de s'inspirer du modèle Akka pour développer un framework dédié aux applications distribuées en utilisant Spring Boot.
- Ce framework d'agents distribués est composé d'acteurs, des entités capables d'envoyer, de recevoir et de traiter des messages, et pouvant communiquer avec d'autres acteurs au sein d'un même microservice ou entre différents microservices.
- Un autre objectif du projet est de développer un système résilient (avec une gestion des pannes) et élastique (capable de s'adapter à la charge).

État de l'art — Modèle des acteurs (Akka)

- Framework de référence pour les systèmes concurrents et distribués.
- Basé sur le **modèle des acteurs**.
- Acteurs indépendants, avec état propre et communication par messages.
- Isolation des erreurs et supervision hiérarchique.

Akka est utilisé comme source d'inspiration conceptuelle, et non comme technologie.

État de l'art — Spring Boot

- Framework Java pour le développement de microservices.
- Auto-configuration et intégration native avec l'écosystème Spring.
- Utilisation des annotations suivantes :
 - ▶ **@SpringBootApplication** : point d'entrée des services.
 - ▶ **@RestController** : exposition des API REST.
 - ▶ **@Service, @Component** : injection de dépendances et structuration du code.

État de l'art — Communication distribuée

- **@EnableDiscoveryClient** : découverte dynamique des microservices via Eureka.
- **WebClient** : communication HTTP asynchrone et non bloquante.
- **CompletableFuture** : gestion des traitements asynchrones (patterns tell / ask).
- **Spring Cloud Stream + RabbitMQ** : communication événementielle inter-services.

État de l'art — Robustesse et performance

- **Resilience4j** : tolérance aux pannes (Circuit Breaker, Retry).
- **Virtual Threads (Java 21)** : threads légers pour une meilleure scalabilité.
- **Docker** : conteneurisation des services et reproductibilité de l'environnement.

Présentation du SAF - Objectifs du framework

Le Spring Actor Framework a été conçu dans le but de fournir une infrastructure permettant :

- la création et la gestion d'acteurs distribués,
- la communication asynchrone et synchrone entre acteurs,
- la supervision et la tolérance aux pannes,
- l'intégration avec une architecture microservices Spring Boot.

Le framework se veut générique et réutilisable, indépendamment de l'application CY Land, qui sert uniquement de démonstrateur.

Présentation du SAF - Principes de fonctionnement

Dans le Spring Actor Framework, chaque acteur représente une entité autonome du système. Il dispose :

- d'un identifiant unique,
- d'une boîte aux lettres (mailbox) pour stocker les messages entrants,
- d'un comportement définissant comment les messages sont traités.

Les acteurs ne communiquent jamais directement entre eux, mais uniquement via des références abstraites appelées *ActorRef*. Cette abstraction permet de masquer la localisation réelle de l'acteur et de faciliter la distribution.

Présentation du SAF - Communication entre acteurs

Le framework implémente deux modes de communication principaux :

- **Tell** : communication asynchrone sans retour. L'émetteur envoie un message et poursuit son exécution sans attendre de réponse.
- **Ask** : communication synchrone avec retour, basée sur des objets *CompletableFuture*, permettant d'obtenir une réponse de l'acteur destinataire.

Ce double mécanisme permet d'adapter la communication aux besoins métier, en privilégiant la performance ou la synchronisation selon les cas.



Présentation du SAF - Supervision et gestion des pannes

Le SAF intègre un mécanisme de supervision inspiré d'Akka. Les acteurs peuvent être surveillés par d'autres acteurs, appelés superviseurs, capables de réagir en cas d'erreur.

Plusieurs stratégies sont disponibles, notamment :

- **OneForOne** : seul l'acteur fautif est impacté.
- **AllForOne** : une erreur affecte l'ensemble d'un groupe d'acteurs.

Ces stratégies permettent d'améliorer la résilience du système et d'éviter la propagation incontrôlée des erreurs.

CY Land : Vue d'ensemble de l'architecture

L'application CY Land repose sur une architecture microservices distribuée, construite autour de plusieurs services Spring Boot indépendants. Cette architecture permet de séparer clairement les responsabilités, de faciliter la maintenance et d'améliorer la scalabilité globale du système.

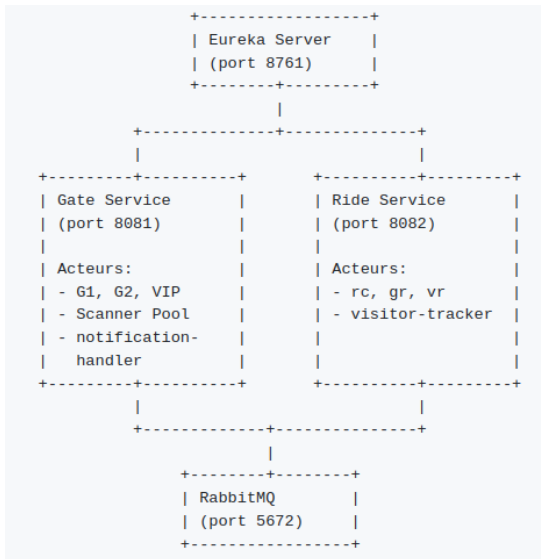
L'architecture se compose principalement de :

- un serveur Eureka chargé de la découverte des services,
- un Gate Service, responsable de la gestion de l'entrée du parc,
- un Ride Service, chargé de la gestion des attractions,
- un broker de messages RabbitMQ, utilisé pour la communication événementielle,
- le Spring Actor Framework, intégré dans chaque microservice.

Chaque micro service héberge ses propres acteurs et communique avec les autres services via des mécanismes de découverte et de messagerie.



CY Land : Vue d'ensemble de l'architecture



CY Land : Gate Service

Le Gate Service représente la porte d'entrée du parc d'attractions. Il est chargé de :

- la validation des tickets des visiteurs,
- le contrôle des accès,
- la réception des notifications provenant des attractions.

Chaque porte est modélisée par un acteur, capable de traiter les demandes de scan de tickets de manière asynchrone ou synchrone. Le Gate Service contient également un acteur dédié à la réception des notifications inter-services, permettant de réagir aux événements provenant du Ride Service (par exemple une panne d'attraction).

CY Land : Ride Service

Le Ride Service est responsable de la gestion des attractions du parc. Il prend en charge :

- la gestion des files d'attente,
- l'entrée des visiteurs dans les manèges,
- le déroulement des cycles d'attraction,
- la gestion des pannes et des réparations.

Chaque attraction est représentée par un acteur, garantissant une gestion isolée de son état. Cette approche permet d'éviter les conflits liés à l'accès concurrent aux données et facilite la gestion des événements asynchrones.

CY Land : Communication inter services

La communication entre micro services s'appuie sur trois mécanismes complémentaires :

- Eureka, pour la découverte dynamique des services,
- WebClient, pour les échanges HTTP asynchrones,
- RabbitMQ, pour la diffusion d'événements entre services.

Implémentation technique : Structure du projet

```
cy-land/
|-- pom.xml                # POM parent
|-- docker-compose.yml
|-- README.md
|-- docs/
|   |-- ARCHITECTURE.md
|   +-- REFERENCES.md
|-- postman/
|   +-- CY_Land_API_Collection.postman_collection.json
|-- actor-framework/      # Framework réutilisable
|   +-- src/main/java/com/music/actor/
|       |-- core/         # Interfaces (Actor, ActorRef, Message...)
|       |-- runtime/       # Implementations
|       |-- supervision/   # Stratégies de supervision
|       |-- scalability/   # Auto-scaling
|       |-- resilience/    # Configuration Resilience4j
|       +-- logging/       # Logs
|-- eureka-server/        # Service Discovery
|-- gate-service/         # Microservice Portes
|   +-- src/main/java/com/music/gate/
|       |-- api/          # Contrôleurs et Services
|       |-- domain/       # Acteurs (GateActor, NotificationHandlerActor)
|       +-- messaging/    # Éditeurs RabbitMQ
+-- ride-service/         # Microservice Attractions
    +-- src/main/java/com/music/ride/
        |-- api/          # Contrôleurs et Services
        |-- domain/       # Acteurs (RideActor)
        +-- messaging/    # Éditeurs et Consommateurs RabbitMQ
```



Implémentation technique : Le Runtime - Threads Virtuels et Mailbox

- **Virtual Threads (Java 21)** : Utilisation d'un `ThreadPoolExecutor` avec des threads virtuels. Cela permet de démultiplier le nombre d'acteurs sans saturer la mémoire.
- **Modèle de Mailbox** : Chaque acteur possède sa propre `LinkedBlockingQueue`.
- **Traitement Séquentiel** : Une boucle de traitement (*processLoop*) garantit qu'un seul message est traité à la fois par acteur, assurant l'isolation de l'état sans verrous (locks) explicites.
- **Cycle de vie** : Gestion fine des états via l'énumération `ActorState` (*RUNNING*, *BLOCKED*, *RESTARTING*, etc.).



Implémentation technique : Communication Distante (`RemoteActorRef`)

- **Abstraction Locale/Distante** : L'utilisation de l'interface `ActorRef` permet à un acteur d'envoyer un message sans savoir si le destinataire est local ou sur un autre service.
- **Service Discovery (Eureka)** : Le système utilise *DiscoveryClient* pour localiser dynamiquement les instances de microservices (ex : *gate-service* vers *ride-service*).
- **Transport HTTP Asynchrone** : Utilisation de *WebClient* (Spring WebFlux) pour des appels non-bloquants vers les endpoints `/actors/{id}/tell` ou `/ask`.

Implémentation technique : Résilience

Inter-Services

- **Circuit Breaker** : Intégré dans `RemoteActorRef` pour stopper les appels vers un service défaillant après 5 échecs, évitant les cascades de pannes.
- **Retry avec Backoff** : Ré-exécution automatique des messages échoués (3 tentatives) avec un délai exponentiel (500ms, 1s, 2s).
- **Patterns Tell/Ask** :
 - ▶ **Tell** : Fire-and-forget asynchrone.
 - ▶ **Ask** : Retourne une `CompletableFuture` avec gestion du timeout.

Implémentation technique : Élasticité - Auto-Scaling Pool

- **Pool Dynamique** : Implémentation de `AutoScalingActorPool` pour les tâches lourdes (ex : scanners de tickets).
- **Mesure de Charge** : Le pool surveille la taille des *mailboxes* des acteurs pour détecter une surcharge.
- **Décision Scaling** :
 - ▶ **Scale-up** : Création d'instances si la charge dépasse 70%.
 - ▶ **Scale-down** : Suppression d'instances si la charge descend sous 20%.
- **Distribution** : Répartition des messages en *Round-Robin* entre les acteurs du pool.

Implémentation technique : Supervision et Persistance

- **Stratégies de Supervision :**
 - ▶ *OneForOne* : Seul l'acteur en erreur est redémarré (ex : Portes).
 - ▶ *AllForOne* : Tout le groupe est redémarré pour garder la cohérence (ex : Attractions).
- **Mécanisme de Snapshot** : Utilisation des méthodes `snapshot()` et `restore()` dans `ActorContext` pour sauvegarder l'état interne avant un crash.
- **Logging Asynchrone** : `DefaultActorLogger` écrit les activités en JSONL via un thread dédié pour ne pas ralentir le runtime.

Résultats et Tests - Tests unitaires et analyse

Capture d'écran 1

```
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.072 s -- in  
frameworkTest  
[INFO]  
[INFO] Results:  
[INFO]  
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0  
[INFO]  
[INFO]  
[INFO] -----< com.music:eureka-server >-----  
[INFO] Building Eureka Server 1.0.0-SNAPSHOT [3/5]  
[INFO]    from eureka-server/pom.xml  
[INFO] -----[ jar ]-----
```

Résultats et Tests - Tests unitaires et analyse

Capture d'écran 1 Explications des résultatst :

La capture d'écran 1 montre le succès des 8 tests unitaires critiques du module actor-framework. Ces tests valident :

- Creation d'acteur
- Pattern tell (asynchrone)
- Pattern ask (synchrone)
- Ordre FIFO des messages
- Blocage/deblocage
- Supervision et redemarrage
- Arret d'acteur



Résultats et Tests - Tests unitaires et analyse

Capture d'écran 1

Explications des résultats :

Analyse technique :

Stabilité : Le framework affiche "Tests run : 8, Failures : 0", garantissant que la logique de supervision (Restart/Stop) et la gestion de l'état interne des acteurs sont fiables avant le déploiement des services métiers.

Performance : Les tests s'exécutent en environ 2 secondes, illustrant l'efficacité des Virtual Threads pour l'initialisation des systèmes d'acteurs.



Résultats et Tests - Tests unitaires et analyse

Capture d'écran 2

```
[INFO] [ticket-scanners-worker-0=====
] [ GATE SERVICE READY
ACTOR_CREATED] Actor created with parent: root
  Gates: 3
  Scanner Pool: 2 workers
  Auto-scaling: ENABLED (2-10 workers)
  Notification Handler: ACTIVE

ENDPOINTS:
- POST /gate/{gateId}/scan?ticketId=XXX
- POST /gate/{gateId}/scan-sync?ticketId=XXX
- GET /gate/{gateId}/status
- POST /gate/{gateId}/block
- POST /gate/{gateId}/unblock
- GET /actors (actor management)
- GET /gate/pool/metrics (scaling metrics)
=====
```



Résultats et Tests - Tests unitaires et analyse

Capture d'écran 2

Initialisation du Gate Service : Cette slide montre que la logique métier est opérationnelle : Le service est marqué comme READY avec " portes et un pool de scanners activé.

On observe que le framework gère dynamiquement entre 2 et 10 workers pour le scan des tickets.

Résilience :

Le Notification Handler est actif, prêt à intercepter les messages d'erreur des autres services (ex : panne de manège).

Interface REST :

Les endpoints exposés permettent de piloter les acteurs (bloquer une porte, demander l'état) de manière synchrone ou asynchrone.



Résultats et Tests - Tests unitaires et analyse

Capture d'écran 3

```
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.344 s -- in com.music.gate.InterServiceIntegrationTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] -----< com.music:ride-service >-----
[INFO] Building Ride Service 1.0.0-SNAPSHOT [5/5]
[INFO]    from ride-service/pom.xml
[INFO] -----[ jar ]-----
```

Cette troisième image montre le succès total des tests d'intégration du gate-service ; Les 18 tests sont passés avec succès. Cela inclut également les tests de communication entre services (InterServiceIntegrationTest).

La réussite des tests d'intégration prouve que les acteurs du Gate Service arrivent à interagir correctement avec l'infrastructure Spring Boot (API REST) et les autres briques du système.



Résultats et Tests - Tests unitaires et analyse

Capture d'écran 4

```
[INFO] [gr] [ACTOR_CREATED] Actor created with parent: root
=====
RIDE SERVICE READY
Attractions: 3

Available rides:
[DEBUG] [vr] [STATE_CHANGE - RC
] - GR
State changed: CREATED -> RUNNING - VR

ENDPOINTS:
- GET /rides
- GET /rides/{id}/state
- POST /rides/{id}/join?ticketId=XXX
- POST /rides/{id}/start-cycle
- POST /rides/{id}/report-fault
- POST /rides/{id}/repair
=====
```



Résultats et Tests - Tests unitaires et analyse

Capture d'écran 4

Le Ride Service est lui aussi opérationnel et prêt à l'emploi, la capture montre le démarrage réussi du Ride Service avec ses trois principales attractions (RC, GR, VR) prêtes à fonctionner.

Chaque attraction est lancée comme un acteur indépendant, passant de l'état CREATED à RUNNING. Le service expose des endpoints pour la gestion des files d'attente et des cycles de fonctionnement.

De plus, l'isolation des acteurs garantit qu'un blocage sur une attraction n'empêche pas les autres de fonctionner.



Résultats et Tests - Bilan global et analyse de l'intégration

```
[INFO] -----
[INFO] Reactor Summary for CY Land - Actor Framework 1.0.0-SNAPSHOT:
[INFO]
[INFO] CY Land - Actor Framework ..... SUCCESS [ 0.617 s]
[INFO] Actor Framework ..... SUCCESS [ 32.891 s]
[INFO] Eureka Server ..... SUCCESS [ 20.514 s]
[INFO] Gate Service ..... SUCCESS [ 41.248 s]
[INFO] Ride Service ..... SUCCESS [ 21.563 s]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 02:02 min
[INFO] Finished at: 2025-12-24T14:27:03Z
[INFO] -----
```

Cette capture d'écran présente le bilan global du projet

Explication des résultats :

Les tests d'intégration simulent le parcours complet d'un visiteur, du scan à l'entrée jusqu'à l'accès aux attractions.

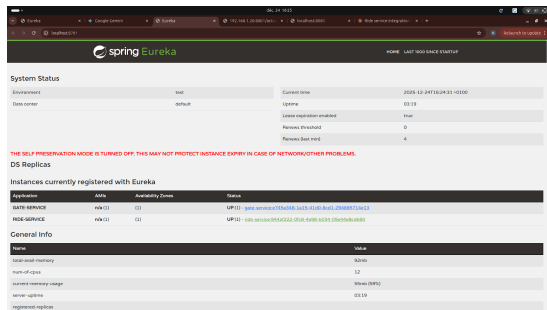


Résultats et Tests - Bilan global et analyse de l'intégration

Voici les résultats :

```
cytechstudent-laptop:/data/Desktop/Documents/ing2/seml/51/CV-Land-Cy-Land/eureka-server$ curl http://localhost:8081/actuator/health
{"status":"UP","components":{"discoveryComposite":{"status":"UP","details":{"services":{"ride-service","gate-service"}},"eureka":{"description":"Remote status from eureka server","status":"UP","details":{"applications":{"RIDE-SERVICE-1","GATE-SERVICE-1"}}},"discoverySpace":{"status":"UP","details":{"total":20485768,"free":15871124928,"refreshed":16483768,"path":"/data/Desktop/Documents/ing2/seml/51/CV-Land-Cy-Land/"},"status":"UP"},"rabbit":{"status":"UP","details":{"version":"3.13.7"},"reactiveDiscoveryClient":{"status":"UP","components":{"simpleReactiveDiscoveryClient":{"status":"UP","details":{"services":{"ride-service","gate-service-1337"},"refreshScope":{"status":"UP"},"ssl":{"status":"UP"},"refreshScope":{"status":"UP"},"ssl":{"status":"UP"}}}}}}}}}}
cytechstudent-laptop:/data/Desktop/Documents/ing2/seml/51/CV-Land-Cy-Land/eureka-server$ curl http://localhost:8082/actuator/health
{"status":"UP","components":{"binders":{"status":"UP","components":{"rabbit":{"status":"UP","details":{"version":"3.13.7"},"discoveryComposite":{"status":"UP","components":{"discoveryClient":{"status":"UP","details":{"services":{"ride-service","gate-service-1337"},"eureka":{"description":"Remote status from eureka server","status":"UP","details":{"applications":{"RIDE-SERVICE-1","GATE-SERVICE-1337"},"discoverySpace":{"status":"UP","details":{"total":20485768,"free":15871124928,"refreshed":16483768,"path":"/data/Desktop/Documents/ing2/seml/51/CV-Land-Cy-Land/"},"status":"UP"},"rabbit":{"status":"UP","details":{"version":"3.13.7"},"reactiveDiscoveryClient":{"status":"UP","components":{"simpleReactiveDiscoveryClient":{"status":"UP","details":{"services":{"ride-service","gate-service-1337"},"refreshScope":{"status":"UP"},"ssl":{"status":"UP"},"refreshScope":{"status":"UP"},"ssl":{"status":"UP"}}}}}}}}}}}}
cytechstudent-laptop:/data/Desktop/Documents/ing2/seml/51/CV-Land-Cy-Land/eureka-server$ curl -X POST "http://localhost:8081/gate/g1/scan?ticketId=7001" POST "http://localhost:8081/gate/g1/scan?ticketId=7001"
{"ticketId":"701","status":"SUCCESS","message":"Gate G2 Status: /cytechstudent-laptop:/data/Desktop/Documents/ing2/seml/51/CV-Land-Cy-Land/eureka-server$ curl -X POST "http://localhost:8081/gate/g2/bisce"
cytechstudent-laptop:/data/Desktop/Documents/ing2/seml/51/CV-Land-Cy-Land/eureka-server$ curl -X POST "http://localhost:8082/rides/r1/join?ticketId=7001"
{"status":"SUCCESS","rideId":"r1c","message":"Added to queue","ticketId":"7001"}cytechstudent-laptop:/data/Desktop/Documents/ing2/seml/51/CV-Land-Cy-Land/eureka-server$
```


Résultats et Tests - Tests d'intégration



The screenshot displays the Spring Cloud Eureka Admin interface. At the top, the 'System Status' section shows environment details (test, default) and current time (2025-12-24T16:24:31+01:00). Below this, a table lists system parameters: 'Uptime' (03:29), 'Service registration enabled' (true), 'Replicas threshold' (0), and 'Replicas lease hold' (4). A red warning message states: 'THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.' The 'DS Replicas' section shows 'Instances currently registered with Eureka' in a table with columns: Application, Attrs, Availability Zones, and Status. Two instances are listed: 'GATE-SERVICE' and 'PDC-SERVICE', both with status 'UP'. The 'General Info' section at the bottom provides details like 'Total available memory' (Admin), 'Number of CPUs' (12), 'Current memory usage' (504MB (50%)), 'Server uptime' (03:19), and 'Registered replicas'.

Application	Attrs	Availability Zones	Status
GATE-SERVICE	id=1	(1)	UP(1) - gate-service-75b2d5b-5a1b-414b-8a1c-2988b71ac13
PDC-SERVICE	id=1	(1)	UP(1) - pdc-service-854f222-0518-8d9b-8d9b-5b6b6b6b6b6b

Name	Value
total-avail-memory	Admin
num-of-cpus	12
current-memory-usage	504MB (50%)
server-uptime	03:19
registered-replicas	

Cette capture présente l'interface d'administration Spring Cloud Eureka qui tourne sur le port 8761. Les tests d'intégration ont permis de valider la communication entre les microservices. Chaque service s'enregistre dynamiquement auprès d'Eureka au démarrage et découvre les autres services sans dépendre d'adresses statiques.

Conclusion

Le projet CY Land a permis de développer un framework solide et résilient, capable de gérer la charge tout en continuant à fonctionner en cas d'erreurs.

Cela nous a permis de maîtriser Spring Boot et les microservices, en construisant des services capables de traiter de nombreux messages en parallèle de manière fluide.

Pour aller plus loin, l'intégration de Kafka permettrait de rendre la communication encore plus robuste et le suivi des visiteurs totalement fiable, même en cas de panne.

Documentation officielle



Spring Boot Reference Documentation, VMware Inc., 2024.

[https:](https://docs.spring.io/spring-boot/docs/current/reference/html/)

[//docs.spring.io/spring-boot/docs/current/reference/html/](https://docs.spring.io/spring-boot/docs/current/reference/html/)



Spring Cloud Netflix Eureka, VMware Inc., 2024.

<https://spring.io/projects/spring-cloud-netflix>



Spring Cloud Stream Reference, VMware Inc., 2024.

<https://spring.io/projects/spring-cloud-stream>



RabbitMQ Documentation, VMware Inc., 2024.

<https://www.rabbitmq.com/documentation.html>



Akka Documentation, Lightbend Inc., 2024.

<https://doc.akka.io/docs/akka/current/index.html>



Resilience4j Documentation, 2024.

<https://resilience4j.readme.io/docs>



Tutoriels et guides

 Spring Guides – Building a RESTful Web Service.
<https://spring.io/guides/gs/rest-service/>

 Spring Guides – Service Registration and Discovery.
<https://spring.io/guides/gs/service-registration-and-discovery/>

 Baeldung – Introduction to Spring Cloud Stream.
<https://www.baeldung.com/spring-cloud-stream>

 Baeldung – Guide to Resilience4j.
<https://www.baeldung.com/resilience4j>

 RabbitMQ Tutorials.
<https://www.rabbitmq.com/tutorials>

Modèle d'acteurs et résilience



B. Storti, *The Actor Model Explained*, 2015.

<https://www.brianstorti.com/the-actor-model/>



M. Fowler, *Circuit Breaker Pattern*, 2014.

<https://martinfowler.com/bliki/CircuitBreaker.html>



C. Hewitt, P. Bishop, R. Steiger, *A Universal Modular Actor Formalism for Artificial Intelligence*, 1973.

<https://dl.acm.org/doi/10.5555/1624775.1624804>

Java 21 et outils

 JEP 444 – Virtual Threads, OpenJDK, 2023.

<https://openjdk.org/jeps/444>

 JEP 441 – Pattern Matching for switch, OpenJDK, 2023.

<https://openjdk.org/jeps/441>

 Oracle Java 21 Documentation.

<https://docs.oracle.com/en/java/javase/21/>

 Docker Documentation.

<https://docs.docker.com/>

 JUnit 5 User Guide.

<https://junit.org/junit5/docs/current/user-guide/>

 Postman Learning Center.

<https://learning.postman.com/docs/getting-started/introduction/>

 Maven Documentation.

<https://maven.apache.org/guides/>



Architecture et supports pédagogiques

-  M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.
-  C. Richardson, *Microservices Patterns*, Manning Publications, 2018.
-  S. Newman, *Building Microservices*, O'Reilly Media, 2021.
-  Cours Spring Boot, Module JEE, CY Tech, 2024–2025.
-  TP Microservices, Module JEE, CY Tech, 2024–2025.